

# VIT BHOPAL UNIVERSITY

## SMART VISION TOOLKIT

### Computer Vision - Project Report

|                  |                       |
|------------------|-----------------------|
| Student          | Akshat Vaidhya        |
| Registration No. | 24BAC10072            |
| Project          | Smart Vision Toolkit  |
| Domain           | Computer Vision       |
| University       | VIT Bhopal University |
| Academic Year    | 2026                  |

Submitted for Academic Project Evaluation

# 1. Introduction

Smart Vision Toolkit is a Python and OpenCV computer vision application that performs a reproducible image-processing pipeline. It generates a synthetic scene and applies grayscale conversion, Gaussian smoothing, Canny edge detection, binary thresholding and contour-based object analysis.

# 2. Problem Statement

Fundamental computer vision operations are often taught as isolated techniques. This project combines them into one workflow so preprocessing, feature extraction and simple object analysis can be observed through actual output images.

# 3. Objectives

- Demonstrate core image-processing operations with OpenCV.
- Create reproducible visual results without an external dataset.
- Detect object boundaries and compute geometric statistics.
- Validate the implementation with automated test cases.
- Maintain a modular project structure.

# 4. Project Scope

The toolkit focuses on basic 2-D image processing. It does not perform deep-learning classification, medical diagnosis, pretrained object recognition or real-time camera tracking. The synthetic input makes the project self-contained.

# 5. Functional Requirements

| Module             | Capability                                                |
|--------------------|-----------------------------------------------------------|
| Input generation   | Create a reproducible sample image with geometric objects |
| Preprocessing      | Convert to grayscale and apply Gaussian blur              |
| Feature extraction | Detect edges using Canny                                  |
| Segmentation       | Create a binary threshold image                           |
| Object analysis    | Find contours, bounding boxes and areas                   |
| Visualization      | Save intermediate and comparison images                   |
| Validation         | Run automated tests for core functions                    |

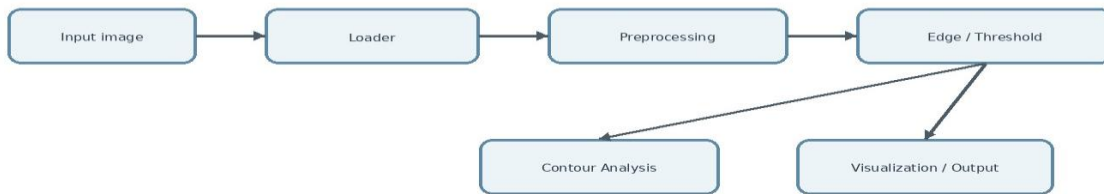
# 6. Technologies and Requirements

Python 3.9+ with OpenCV and NumPy. The main application and tests are separated so individual operations can be verified independently.

| Item                 | Specification       |
|----------------------|---------------------|
| Language             | Python 3.9+         |
| Computer vision      | OpenCV              |
| Numerical processing | NumPy               |
| Interface            | Command line        |
| Input                | Synthetic PNG scene |
| Output               | PNG visualizations  |

# 7. Project Architecture

The processing sequence is: load image -> grayscale -> Gaussian blur -> Canny edges and threshold -> contour analysis -> annotated and comparison outputs.



## 8. Implementation Details

The pipeline module provides reusable functions for image loading, grayscale conversion, Gaussian blur, edge detection, thresholding and contour analysis. Contours smaller than the area threshold are ignored, and each retained object receives area, perimeter and bounding-box values.

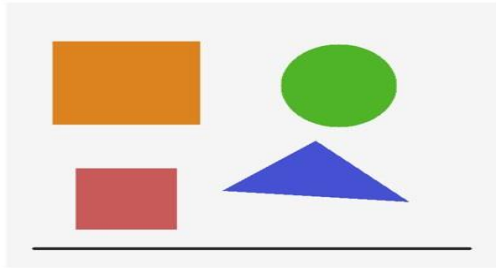
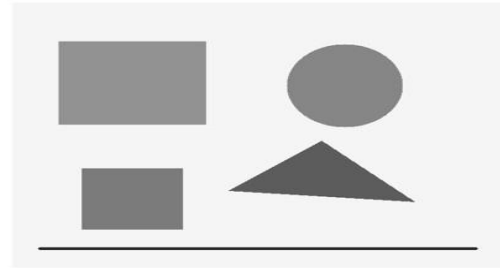
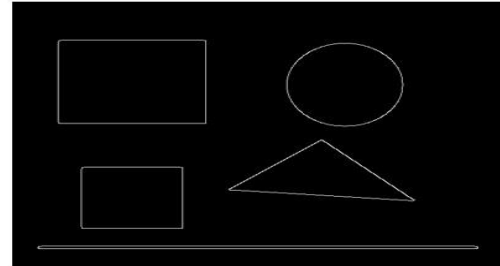
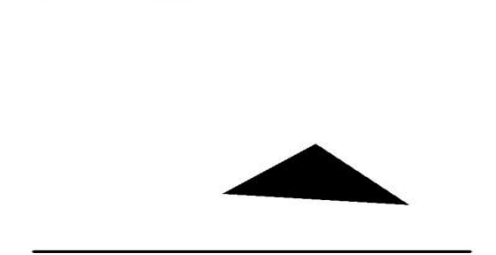
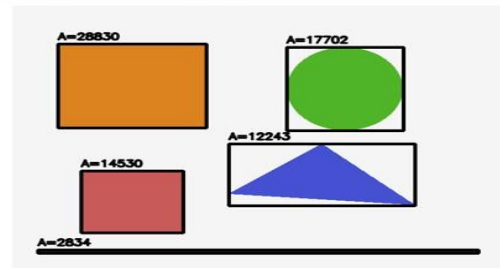
```
load_image(path) to_grayscale(image)
blur_image(gray)
detect_edges(blurred)
threshold_image(gray)
find_objects(edges, original)
run_pipeline(input_path, output_dir)
```

## 9. Execution Workflow



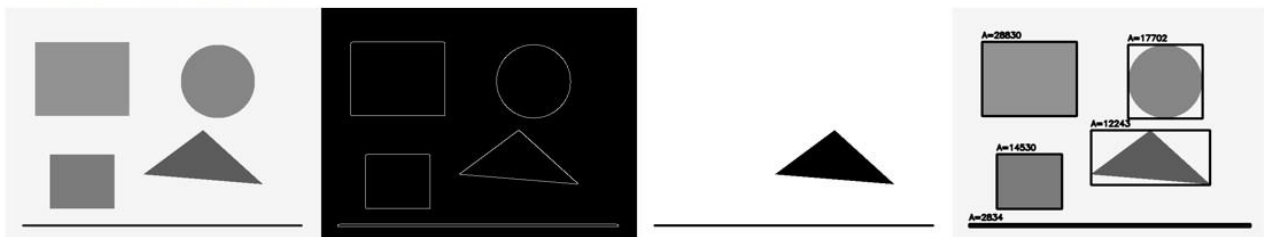
## 10. Output Gallery

Intermediate outputs generated from the same reproducible sample image.

**Original****Grayscale****Gaussian Blur****Canny Edges****Binary Threshold****Contours / Objects**

## 11. Pipeline Comparison

The comparison image places the grayscale, edge, threshold and contour views side-by-side so each processing stage can be checked visually.

**Pipeline Comparison**

## 12. Object Analysis Results

| # | Area    | X   | Y   | Width | Height | Perimeter |
|---|---------|-----|-----|-------|--------|-----------|
| 1 | 28830.0 | 59  | 69  | 192   | 152    | 681.66    |
| 2 | 17702.0 | 355 | 75  | 151   | 151    | 499.24    |
| 3 | 14530.0 | 89  | 299 | 132   | 112    | 481.66    |
| 4 | 12243.0 | 280 | 250 | 241   | 112    | 573.13    |
| 5 | 2834.0  | 33  | 442 | 570   | 6      | 1144.49   |

Image height: 480

**Image width:** 640  
**Channels:** 3  
**Mean grayscale:** 215.57  
**Edge pixels:** 3450  
**Threshold foreground pixels:** 292141

### 13. Testing and Results

Seven automated test cases were executed covering image loading, grayscale conversion, blur, edge detection, thresholding, contour analysis and complete output generation.

| Test Case             | Expected Result                     | Status |
|-----------------------|-------------------------------------|--------|
| TC01 - Load image     | Shape is 480 x 640 x 3              | PASS   |
| TC02 - Grayscale      | Output shape is 480 x 640           | PASS   |
| TC03 - Blur           | Shape preserved after Gaussian blur | PASS   |
| TC04 - Edge detection | Non-zero edge pixels are produced   | PASS   |
| TC05 - Threshold      | Output contains only 0 and 255      | PASS   |
| TC06 - Contours       | At least three objects detected     | PASS   |
| TC07 - Full pipeline  | All seven PNG outputs are created   | PASS   |

Smart Vision Toolkit - Automated Test Results

```
-----
Tests run: 7
Passed: 7
Failed: 0
Errors: 0
OVERALL: PASS
```

```
.....
-----
Ran 7 tests in 0.086s
```

OK

### 14. Challenges and Learnings

The main learning points are understanding smoothing effects on edges, selecting a suitable threshold, filtering insignificant contours and organizing the pipeline into reusable functions. Automated tests provide repeatable validation.

### 15. Limitations and Future Enhancements

The project uses synthetic images and classical techniques. Future work could add camera input, color-space segmentation, Hough transforms, perspective correction, feature descriptors, machine-learning classification, real-time performance measurement and a graphical interface.

### 16. Conclusion and References

Smart Vision Toolkit demonstrates a complete and reproducible classical computer vision workflow using Python, OpenCV and NumPy. The project connects preprocessing, edge extraction, thresholding and object analysis with visual outputs and automated validation.

#### References

- OpenCV documentation.
- NumPy documentation.
- Python documentation.
- VIT Bhopal University project evaluation guidelines.

#### Submission Checklist

- Source code
- requirements.txt
- README.md and statement.md
- Automated test cases
- Output images
- Report PDF